让小模型做好高频决策

Strands Decider & Jev： 从模型范式，到 Decider × GPT 的真实演示

01

认识决策模型

Strands Decider / Jev
面向软件的有限判断

02

走进支付演示

同一业务输入
选择、速度与执行结果

03

形成应用结论

哪些节点适合使用
怎样评估真实收益

模型介绍 → 工作原理 → 支付对比 → 实测结论 · 2026.10.11

软件流程，需要大量小而明确的判断

选哪个队列？调用哪个工具？证据是否足够继续？

接收请求

客户消息 / 当前状态

选择路径

队列 / 工具 / 下一步

检查结果

是否完成 / 是否升级

这些判断的共同点

候选明确、频繁调用、结果被代码消费

决策模型的目标

用受约束的输出，降低判断开销

自然语言理解与路径选择交给模型；金额、权限与执行前提由代码核验。

决策模型：状态输入，类型化答案输出

把一个复杂流程拆成明确问题，再在应用代码中组合答案。

共享输入：业务状态 + 问题 + 候选或判断条件

Choice • 多选一

该交给哪个客服队列?

支付 / 退款 / 商户运营

Noul • 是 / 否

是否表达了退款诉求?

返回 P(true)

Score • 有序评分

这条请求有多紧急?

在定义好的等级上评分

输出类型受约束 ≠ 业务判断必然正确；概率与置信度需要结合业务数据验证。

Jev 与 Strands Decider：同一方向，两种实现

共同强调快速、结构化的判断，以及可供程序使用的概率信息。

Jev / TypeSafe

System One 模型 · 官方 API / SDK
强调校准决策训练（RLCD）
类型化输出与多问题并行评估

Strands Decider / Strands Labs

开源 2B 决策模型 · 代码与权重公开
轻量模型主体 + Pointer Head
面向实验，可本地或自行部署

Jev 为产品背景介绍；本项目实测的是 Strands Decider 与 GPT，没有接入 Jev。

Strands Decider： 用候选评分头完成选择

官方 v19： Qwen3.5-2B-Base + rank-16 LoRA + Pointer Head

输入业务

state / question

criteria / 候选

→

模型主体

Qwen3.5-2B

LoRA 任务适配

→

候选评分

Pointer Head

候选位置 ↔ 答案位置

→

输出决策

选择 / 分布

概率 / 置信信息

对当前候选直接评分，减少逐 token 文本生成

适合明确、聚焦的判断；开放式写作与复杂长链推理，需要不同模型能力。

同样的候选，走两条决策路径

Decider 评分后取最高项； GPT 逐 token 生成候选 ID 。

共同输入：业务事实 + 问题 + 候选条件

DECIDER

编码输入

业务与候选表示

→

Pointer Head

对候选计算分数

→

最高分候选

choice + 概率分布

GPT

读取提示

同一业务与候选

→

生成 token

结构化选择 JSON

→

解析与校验

有效候选 ID

两条路径的输出都交给应用执行器；模型本身不执行转账。

BERT 、 Decider 、 GPT： 适配方式一页看懂

区别在输出头、候选表达与任务范围。

BERT 分类器

固定标签 → 分类头
标签稳定、领域标注
新增类别通常需适配

Decider

请求候选 → 共享评分
条件用自然语言描述
专注有限动作选择

GPT

提示候选 → 生成 JSON
支持开放式生成与规划
也可做受限动作选择

BERT/NLI 也可做动态候选评分；本项目未对 BERT 进行速度或质量实测。

适用场景：候选清晰，结果可检查

从 Agent 路由与守卫，到客服、支付与运营判断。

模型与工具路由

选择模型、工具或业务队列

工具调用检查

参数有无依据，是否需澄清

客服分流与升级

识别诉求，跟进或转人工

策略分类与审核

将自然语言映射到策略

RAG 与内容评估

判断相关性、质量与覆盖

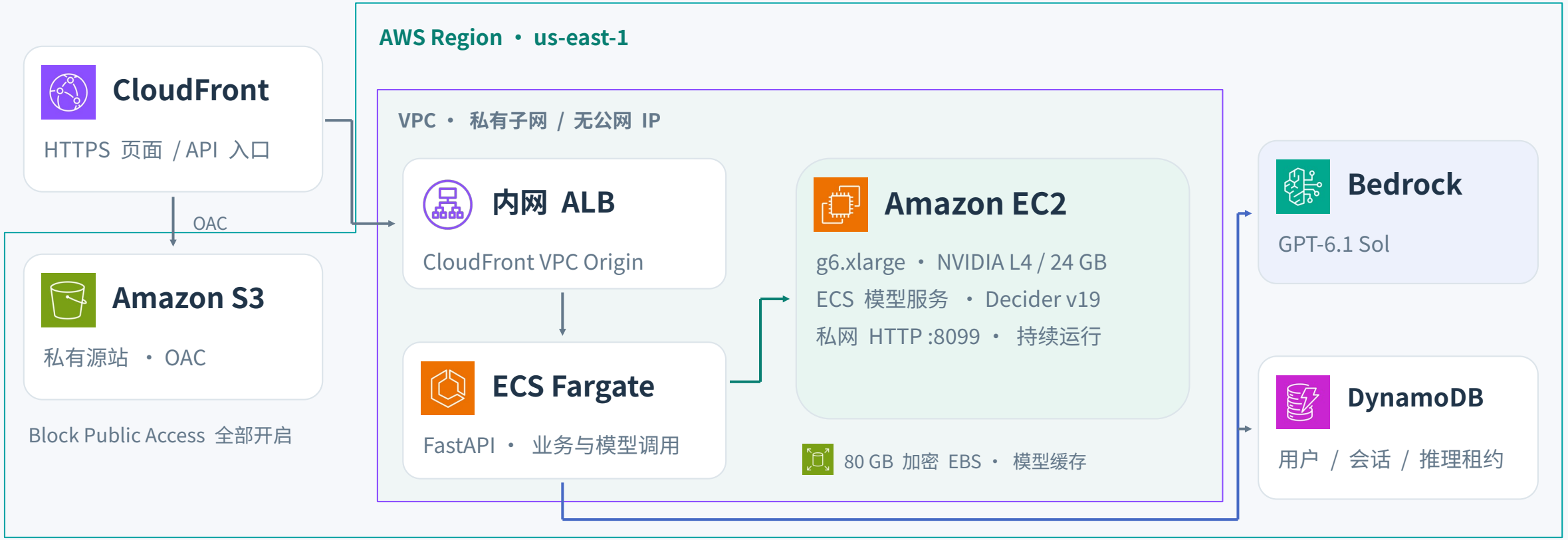
记忆与上下文选择

决定保留或检索哪些片段

接下来用支付业务，把「选择 → 执行 → 结果」放到同一条可观察流程中。

一套业务后端，对接 EC2 与 Bedrock

2026-10-11 当前部署 · EC2 g6.xlarge / L4 · On-Demand 常驻运行



API 直接调用常驻模型；后端保留并发与执行校验。历史测速数据保留原采集环境。

四个支付场景，贯穿五个业务节点

同一笔业务，分别交给 Decider 和 GPT 。

未到账

确认数 3 / 12

少付

100 → 99.80

退款

地址与审批待补齐

结算

到账状态与时效

客服受理

节点 1

→

交易核验

节点 2

→

风控审批

节点 3

→

业务处置

节点 4

→

回复结案

节点 5

客户消息、事实快照与候选条件均可追溯。

比较方法：同一业务输入，独立执行

候选与条件相同；保留两模型的原始选择、耗时和实际业务结果。

同一业务快照 + 同一问题 + 同一候选条件

Decider / EC2

私网 HTTP → EC2 模型服务
返回选择、分布与置信信息
记录耗时，两侧独立执行

GPT / Bedrock

返回候选 ID JSON，校验结果
记录后端决策耗时，独立执行

选择有何差异

每步与累计多快

业务是否完成

速度主统计使用成功同输入配对；反馈执行结果单独观察。

真实界面：同一输入，输出与耗时并排看

USDC 未到账 · 客服受理：两模型都选择「支付客服」。



线上原始截图 · 2026-10-10 · 每轮保留输入、原始选择、实际动作和耗时。

五步对照：耗时差异贯穿每次选择

确认数 3 / 12 ，固定事实保持不变；审批节点出现选择差异。

节点	Decider	GPT
客服受理	支付客服 · 91.8 ms	支付客服 · 1.17 s
交易核验	等待链上确认 · 94.9 ms	等待链上确认 · 1.33 s
风控审批	等待人工审批 · 87.4 ms	规则检查通过 · 3.77 s
业务处置	挂起并跟进 · 91.0 ms	挂起并跟进 · 0.70 s
回复结案	回复并持续跟进 · 93.2 ms	回复并持续跟进 · 0.82 s
	累计 0.458 s	累计 7.792 s

累计差约 17.00× ；这是该案例 5 次成功同输入决策的单次观测。

同一审批问题，为什么选择不一样?

关键条件：这是收款场景，只有退款才需要退款审批。

相同业务输入 收款未到账 • 无风险标记 • 确认数 3/12 • refund_approved=false

候选条件：普通收款无风险 → 检查通过；退款才检查地址与人工审批。

Decider • 87.4 ms

候选评分 → 取最高分

选择「等待人工审批」

等待审批 58.54% > 检查通过 18.49%

本例增加了不需要的审批。

GPT • 3,771.8 ms

按候选条件生成 JSON

{"choice": "approved"}

界面标签：「规则检查通过」

本次选择符合“非退款无需退款审批”。

为何会不同? 自然语言条件由模型判断，并非程序 if/else 。

可能干扰：退款审批字段在本例不适用；内部原因尚不能确认。候选概率表示偏向，不是正确率。

业务结果：检查通过 ≠ 已到账；两侧仍等待确认达标， Decider 另有待审批状态。

处理 5/5 步，资金仍在等待确认

两模型最终都挂起并跟进；确认条件尚未满足。

完成选择

各返回 5 个动作

挂起资金

确认数仍为 3 / 12

持续跟进

工单未结案



完成标准：目标订单 + 对应账本 + 工单结案

下一步需要外部证据更新；反复调用模型不会增加链上确认数。

退款差异：模型想执行，条件是否允许?

地址未验证、审批待完成：两模型建议不同，实际资金动作都挂起。



Decider： 提交模拟退款 → 应用挂起； GPT： 直接挂起。执行保护不能计作模型判断正确。

继续推进：证据更新后，再判断与执行

条件不足时等待；把最新事实和执行结果交给下一次决策。



USDC 未到账

模拟确认数： 3 / 12 → 12 / 12

稳定币退款

模拟验证地址 → 模拟审批通过

同一套机制，各侧独立推进；外部事件由模拟器更新。

执行结果：两组业务，双方均完成

完成标准：目标订单、对应账本与工单结案。

场景	Decider	GPT	每侧决策数	业务结果
USDC 未到账	0.570 s	7.269 s	6 次	双方完成
稳定币退款	0.897 s	10.542 s	9 次	双方完成

退款案例 • 实际执行结果



后端累计决策耗时，不含外部事件等待；两侧后续输入可能不同，不计入同输入速度基准。

19 组成功配对，累计决策耗时相差 12.20×

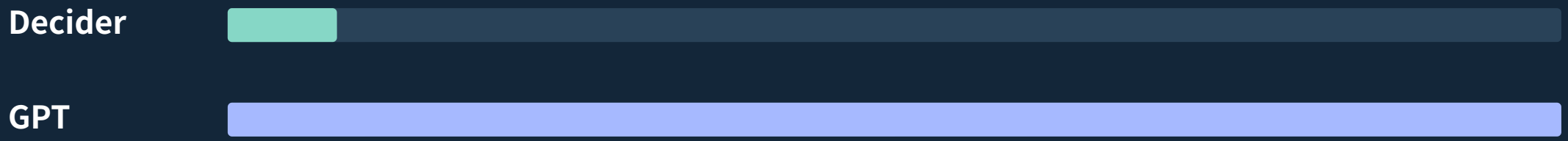
同一批业务输入与候选 · 每侧 20 次尝试，其中 19 组成功配对

1.810 s

Decider · 19 次累计

22.068 s

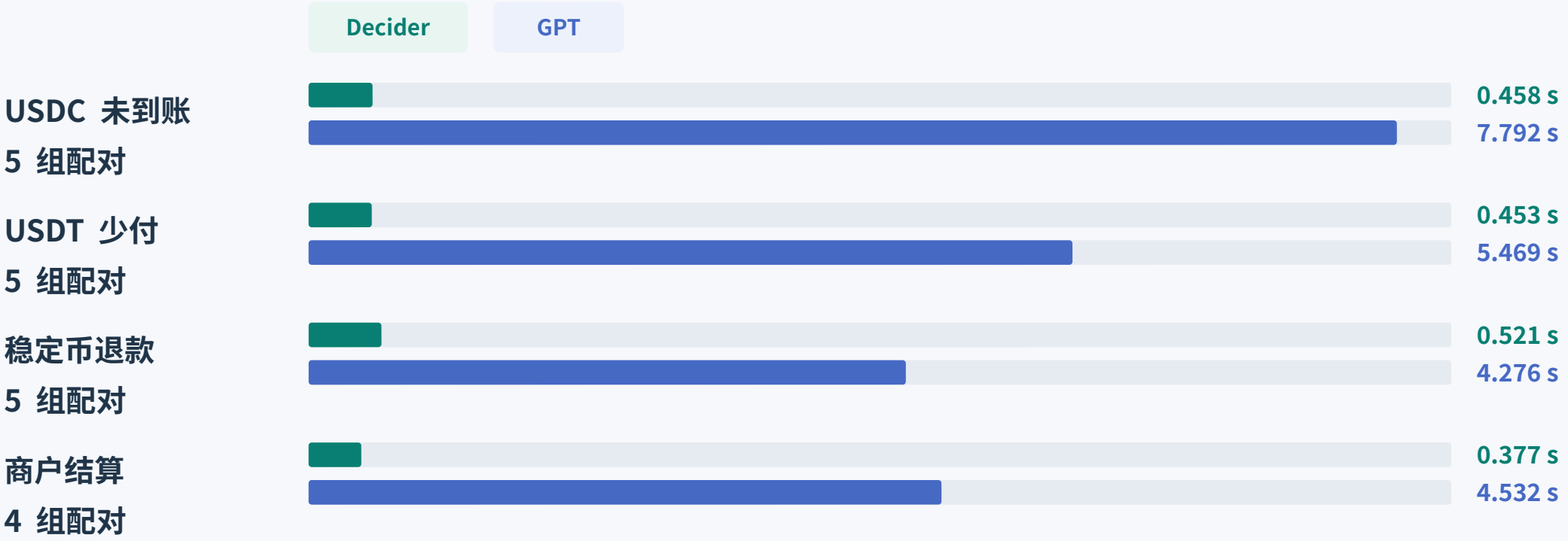
GPT · 19 次累计



单次观测；另有 1 次 GPT 读取超时，未计入成功耗时。不是全程用时或稳定性能保证。

按场景比较：相同输入下的累计耗时

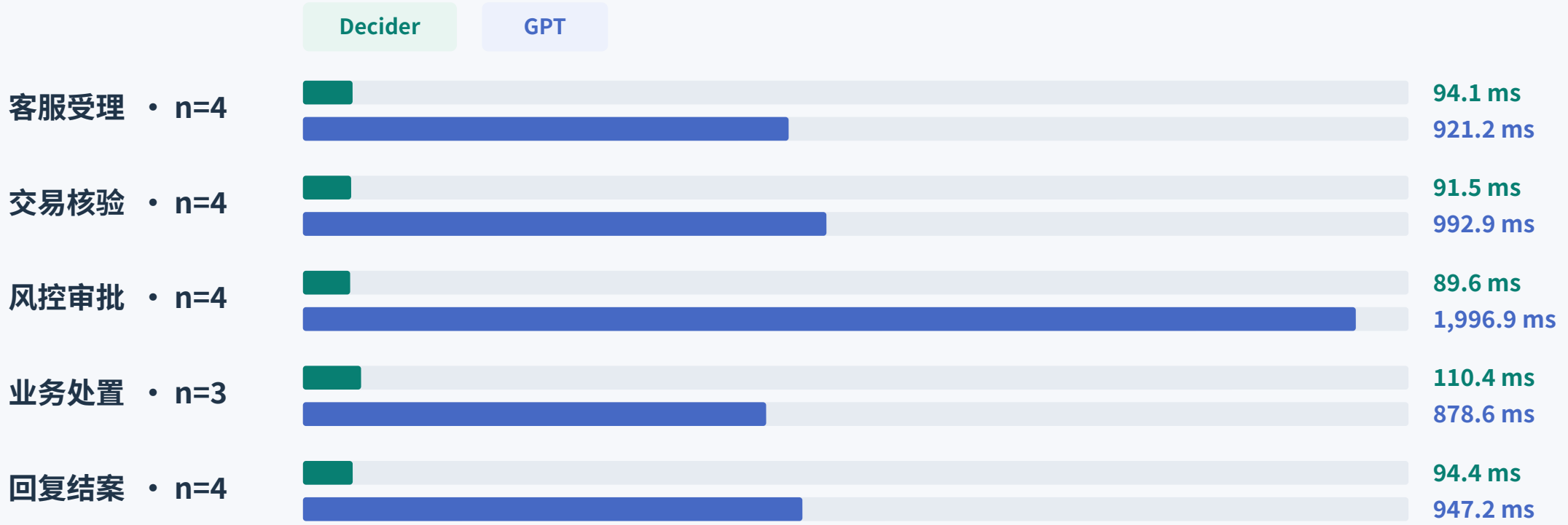
同一场景中，两侧使用完全相同的成功配对子集。



结算仅含 4 对成功请求，不能当作完整五步运行耗时；跨场景比较需留意请求数。

按节点比较：每次判断平均花多久

每个节点仅统计成功同输入配对，横轴为平均毫秒数。



业务处置仅 3 组成功配对；商户结算的 GPT 处置请求超时。条形均从 0 起。

本轮对比：速度、规则选择与实际执行

四个案例 × 五个步骤，每侧调用 20 次；各项数字衡量不同的事情。

比较什么	Decider	GPT	怎么理解
累计决策耗时	**1.810 秒**	**22.068 秒**	19 组同输入成功配对的累计时间 失败等待未计入这两个数字
调用是否成功	**20 / 20 次**	**19 / 20 次**	GPT 有 1 次读取超时 返回了选择，不代表业务已经完成
与业务规则一致	**13 / 19 次**	**19 / 19 次**	选择与开发者预设规则一致的次数 这不是通用准确率
商户结算执行	**处理完 5 步 结算仍待跟进**	**方案未执行 处置步骤超时**	GPT 其余 4 步虽返回选择 但缺少完整方案，本次未进入沙盒执行

为什么比 19 组？ 20 组尝试 − 1 组 GPT 超时；双方使用同一组有效记录。

规则例：普通收款无风险标记，应选「检查通过」； Decider 那次选「等待人工审批」， GPT 选「检查通过」。

仅描述本轮测试；决策速度、规则符合度和业务完成是不同指标。

在支付流程中，按任务分工组合使用

以历史观测为起点，继续按真实业务分布评测。

业务请求

消息 + 当前事实

→

任务分流

范围 / 复杂度 / 风险

→

模型建议

Decider 或 GPT

→

执行系统

校验与落实动作

Decider

队列路由、受限候选、高频下一步选择

GPT

复杂语义、开放式内容、异常分析

组合方案为应用建议；当前演示以两模型并排对比为主。

总结：用合适的模型，完成明确的判断

从 Jev 与 Decider 的模型方向，回到本项目的真实观察。



模型方向

有限输出，聚焦判断
Jev 与 Decider 的共同方向



历史实测

Decider 决策更快
选择差异与失败同样可见



应用落地

模型给建议，系统做校验
以真实业务结果检验收益

下一步：业务数据、重复采样、概率校准、吞吐与整体成本。

打开 Demo，查看代码与实测记录

从同一业务输入开始，观察两模型的选择、速度与执行结果。

在线 Demo

https://d3i7ogfx5ml7dc.cloudfront.net/#crypto

加密支付业务台 · 使用现有账号登录

GitHub Repo

https://github.com/milan9527/StrandsDecider

架构、代码、演示资料与实测记录 · 需仓库权限

官方来源： Strands Decider 博客 / TypeSafe System One 文档 · 详见逐页讲稿。